

Homework 3 Joseph Shin

Joseph Shin

2025-09-04

Contents

EXPLANATIONS ARE IN THE OTHER PDF FILE

```
library(outliers)
```

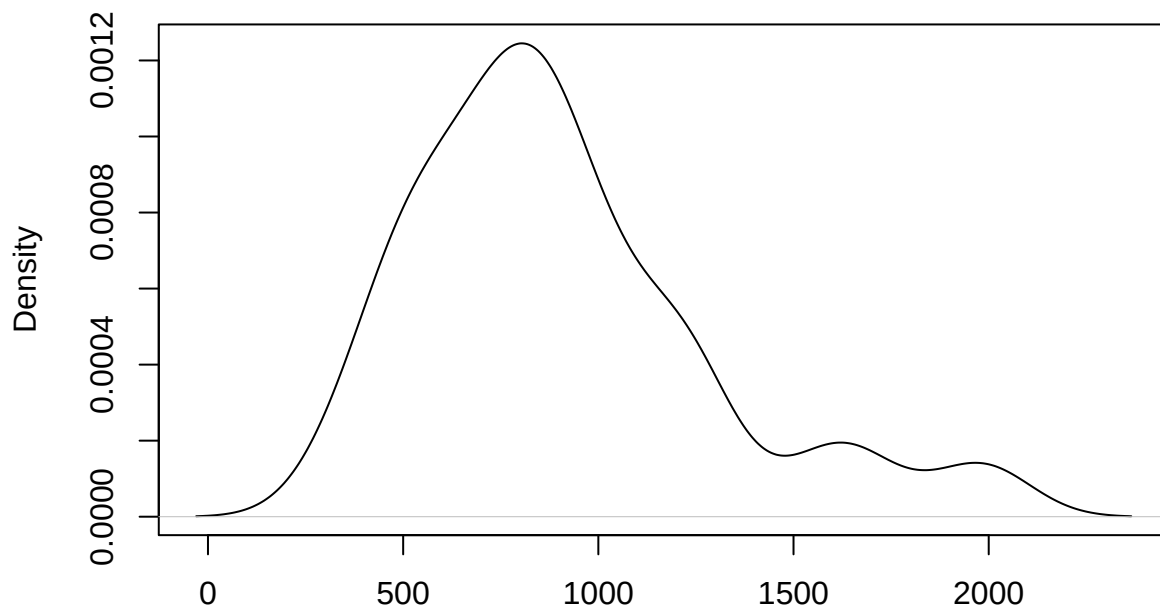
```
## Warning: package 'outliers' was built under R version 4.3.3
```

```
library(ggplot2)
```

```
## Warning: package 'ggplot2' was built under R version 4.3.3
```

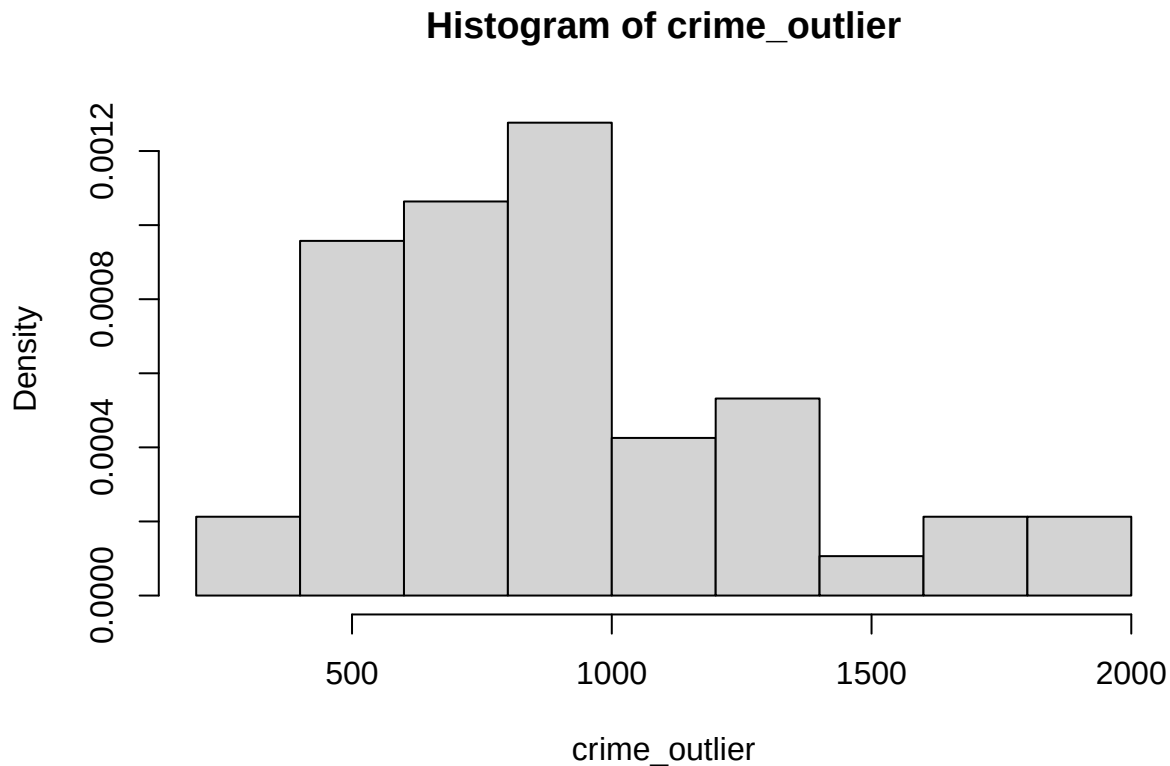
```
crime = read.csv("uscrime.txt", sep = '\t')  
crime_outlier = crime[,16]  
plot(density(crime_outlier))
```

density(x = crime_outlier)



N = 47 Bandwidth = 124.1

```
hist(crime_outlier, freq = FALSE)
```



```
shapiro.test(crime_outlier)
```

```
##  
##  Shapiro-Wilk normality test  
##  
## data:  crime_outlier  
## W = 0.91273, p-value = 0.001882
```

Looking at this histogram of the number of crimes, the distribution is not normal, but instead, is right skewed. Additionally, according to the Shapiro test, the p-value is less than 0.05, so we can reject the null hypothesis and conclude that the distribution is not normal. Additionally, in the density curve, you can see the right skew. The right skew may mean that there are potential outliers in the higher values. The `grubbs.test()` is performed under the assumption that the distribution is normal. Since it is not normal, we may have to include different methods to identify the outliers. However, we can still use a for loop to run a `grubbs.test` to identify one outlier, remove it, and then run the `grubbs.test()` until the distribution becomes normal.

Use the `grubbs` function to take out one outlier, and then test the `grubbs` function again, and keep doing it until there are no outliers left.

```
removed_outliers <- c()  
  
repeat {  
  # Test for normality  
  p_normality <- shapiro.test(crime_outlier)$p.value
```

```

if (p_normality > 0.05) {
  cat("Data is now approximately normal. Stopping.\n")
  break
}

# Run Grubbs' test
g_test <- grubbs.test(crime_outlier, type = 10)
print(g_test)

# Extract the outlier
outlier_val <- as.numeric(strsplit(g_test$alternative, " ")[[1]][3])

if (!is.na(outlier_val)) {
  cat("Removing outlier:", outlier_val, "\n")
  removed_outliers <- c(removed_outliers, outlier_val)
  crime_outlier <- crime_outlier[crime_outlier != outlier_val]
} else {
  cat("No more outliers found. Stopping.\n")
  break
}
}

```

```

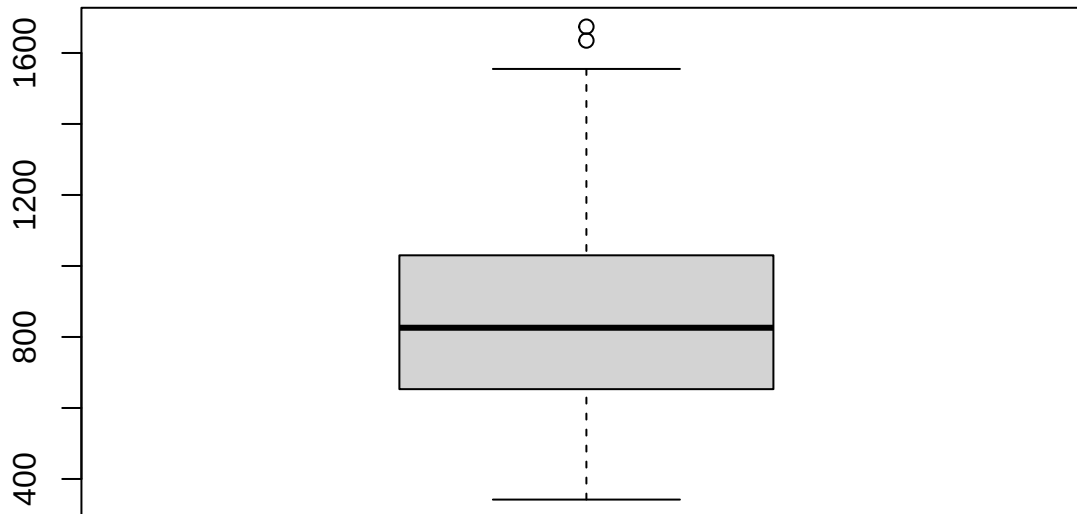
##
## Grubbs test for one outlier
##
## data: crime_outlier
## G = 2.81287, U = 0.82426, p-value = 0.07887
## alternative hypothesis: highest value 1993 is an outlier
##
## Removing outlier: 1993
##
## Grubbs test for one outlier
##
## data: crime_outlier
## G = 3.06343, U = 0.78682, p-value = 0.02848
## alternative hypothesis: highest value 1969 is an outlier
##
## Removing outlier: 1969
## Data is now approximately normal. Stopping.

```

```

bp_crime = boxplot(crime_outlier)

```



According to the boxplot, the maximum value is 1,635. However, there are three values higher than the maximum that are potential outliers: 1674, 1969, and 1993. We can use the boxplot outlier formula to see if these points are considered outliers or not. Since $Q3 = 1057.5$ and $Q1 = 658$, the $IQR = Q3 - Q1 = 399.5$. Then, we use the equation $Q3 + 1.5 * IQR = 1057.5 + 1.5 * 399.5 = 1656.75$. Since 1675, 1969, and 1993 are all greater than 1656.75, we can consider these three points outliers. Based on the boxplot, there does not seem to be an outlier that is lower than the minimum. Therefore, I decided to use type 20 because we would test two outliers in one tail (higher side).

```
temps = read.delim('temps.txt', check.names = FALSE)
temps
```

##	DAY	1996	1997	1998	1999	2000	2001	2002	2003	2004	2005	2006	2007	2008
## 1	1-Jul	98	86	91	84	89	84	90	73	82	91	93	95	85
## 2	2-Jul	97	90	88	82	91	87	90	81	81	89	93	85	87
## 3	3-Jul	97	93	91	87	93	87	87	87	86	86	93	82	91
## 4	4-Jul	90	91	91	88	95	84	89	86	88	86	91	86	90
## 5	5-Jul	89	84	91	90	96	86	93	80	90	89	90	88	88
## 6	6-Jul	93	84	89	91	96	87	93	84	90	82	81	87	82
## 7	7-Jul	93	75	93	82	96	87	89	87	89	76	80	82	88
## 8	8-Jul	91	87	95	86	91	89	89	90	87	88	82	82	90
## 9	9-Jul	93	84	95	87	96	91	90	89	88	89	84	89	89
## 10	10-Jul	93	87	91	87	99	87	91	84	89	78	84	86	87
## 11	11-Jul	90	84	91	82	96	90	84	84	90	83	90	85	89
## 12	12-Jul	91	88	86	77	93	90	77	86	89	86	91	87	93
## 13	13-Jul	93	86	88	73	91	86	82	87	91	84	91	86	85
## 14	14-Jul	93	90	87	81	93	82	88	84	91	87	91	84	88
## 15	15-Jul	82	91	91	81	93	82	91	86	84	84	91	81	89
## 16	16-Jul	91	91	87	86	93	84	93	88	84	85	91	86	89
## 17	17-Jul	96	89	90	82	91	87	93	88	84	89	93	89	88
## 18	18-Jul	95	89	91	87	97	88	93	88	87	90	93	89	90
## 19	19-Jul	96	89	95	88	100	90	93	88	84	89	96	88	91
## 20	20-Jul	99	90	91	90	99	87	91	88	88	89	93	86	94
## 21	21-Jul	91	89	91	90	93	84	95	89	89	90	93	86	95
## 22	22-Jul	95	84	89	91	96	87	91	86	89	91	91	79	92
## 23	23-Jul	91	87	91	93	87	90	89	81	93	91	86	82	87
## 24	24-Jul	93	88	91	93	82	84	87	82	95	90	87	87	88
## 25	25-Jul	84	89	86	91	75	82	84	84	89	92	88	87	89

## 26	26-Jul	84	89	88	93	82	88	86	87	87	94	93	87	87
## 27	27-Jul	82	91	80	93	88	90	89	87	84	92	95	90	90
## 28	28-Jul	79	91	88	93	91	84	91	89	89	90	96	89	93
## 29	29-Jul	90	89	89	93	89	89	91	88	87	83	91	87	92
## 30	30-Jul	91	88	90	97	87	89	88	84	89	78	91	92	90
## 31	31-Jul	87	72	86	99	86	87	90	88	90	84	94	90	88
## 32	1-Aug	86	80	86	96	86	84	93	84	91	82	95	92	89
## 33	2-Aug	90	84	82	93	81	84	91	84	90	86	95	92	92
## 34	3-Aug	84	88	84	88	84	84	91	84	91	88	97	94	91
## 35	4-Aug	91	89	86	89	88	86	91	82	91	91	98	97	91
## 36	5-Aug	93	88	90	91	91	88	93	84	90	88	96	96	92
## 37	6-Aug	88	84	89	93	91	84	97	82	84	86	89	98	94
## 38	7-Aug	91	84	89	93	91	86	87	84	81	80	97	98	90
## 39	8-Aug	84	80	86	93	91	88	87	84	82	82	96	100	86
## 40	9-Aug	90	73	82	91	96	87	86	86	84	85	95	103	85
## 41	10-Aug	89	80	87	90	95	88	88	87	75	83	96	103	85
## 42	11-Aug	88	86	88	96	89	86	89	84	82	87	88	100	88
## 43	12-Aug	86	88	84	98	89	86	91	81	80	88	84	90	81
## 44	13-Aug	84	88	86	97	89	81	91	87	77	86	81	100	81
## 45	14-Aug	86	87	80	98	89	87	89	89	82	90	87	99	84
## 46	15-Aug	89	88	82	93	94	84	88	90	82	92	86	102	87
## 47	16-Aug	90	91	86	93	97	90	90	86	84	89	89	101	86
## 48	17-Aug	91	91	84	96	99	91	91	89	86	90	86	101	85
## 49	18-Aug	91	89	87	98	101	91	93	90	86	90	88	97	86
## 50	19-Aug	90	89	90	98	101	87	91	90	89	89	88	95	90
## 51	20-Aug	89	88	79	89	97	86	93	87	88	92	93	96	90
## 52	21-Aug	90	82	84	91	87	88	93	88	82	94	91	99	85
## 53	22-Aug	91	79	87	91	86	90	91	88	84	93	88	104	82
## 54	23-Aug	91	81	87	90	88	88	95	90	84	87	87	98	78
## 55	24-Aug	91	82	88	80	92	93	93	89	87	85	83	95	83
## 56	25-Aug	84	84	90	82	92	90	91	88	82	84	85	94	78
## 57	26-Aug	88	87	91	89	90	91	88	89	86	84	88	92	83
## 58	27-Aug	84	90	89	88	90	91	84	90	88	86	88	88	80
## 59	28-Aug	86	90	90	90	92	81	82	91	90	86	90	88	86
## 60	29-Aug	88	91	93	91	92	86	82	89	87	85	90	89	89
## 61	30-Aug	84	91	93	91	88	81	78	88	88	85	88	89	89
## 62	31-Aug	82	88	91	84	87	82	77	89	87	85	80	86	88
## 63	1-Sep	80	88	87	88	79	80	84	88	82	85	85	84	81
## 64	2-Sep	73	91	84	91	81	75	84	86	80	88	86	83	85
## 65	3-Sep	87	93	77	84	82	73	89	87	81	87	85	88	83
## 66	4-Sep	84	81	90	93	87	81	95	87	82	85	88	91	85
## 67	5-Sep	87	81	91	96	81	90	93	84	84	81	83	89	88
## 68	6-Sep	89	82	89	96	66	88	91	73	81	81	85	85	87
## 69	7-Sep	89	86	90	91	66	87	88	75	86	83	80	86	89
## 70	8-Sep	89	88	89	91	75	86	87	81	73	85	83	88	90
## 71	9-Sep	91	84	79	77	80	86	91	82	84	86	83	89	88
## 72	10-Sep	84	80	78	87	82	89	95	79	84	84	85	89	87
## 73	11-Sep	86	82	81	87	84	87	95	80	84	84	84	89	83
## 74	12-Sep	88	86	84	87	86	84	90	81	81	86	82	86	87
## 75	13-Sep	78	87	89	86	87	84	75	84	79	88	70	85	86
## 76	14-Sep	79	87	87	87	86	86	78	82	79	88	80	81	88
## 77	15-Sep	86	88	87	89	80	77	91	82	73	91	82	82	79
## 78	16-Sep	82	88	88	81	75	77	88	81	75	88	83	76	80
## 79	17-Sep	82	90	87	81	73	81	86	81	80	86	85	78	69

## 80	18-Sep	78	88	82	82	73	81	81	81	79	88	85	79	82
## 81	19-Sep	79	91	80	79	84	82	80	84	78	90	79	82	81
## 82	20-Sep	79	95	82	68	87	84	86	87	73	90	73	81	79
## 83	21-Sep	78	89	82	79	77	86	84	82	75	90	75	78	75
## 84	22-Sep	81	70	88	72	73	87	77	75	80	86	82	86	84
## 85	23-Sep	84	80	84	75	81	88	82	81	84	87	86	83	82
## 86	24-Sep	84	82	81	78	84	69	73	80	82	88	84	89	78
## 87	25-Sep	87	66	82	81	82	66	69	82	81	85	75	87	82
## 88	26-Sep	84	70	84	82	68	72	75	82	79	77	78	84	80
## 89	27-Sep	79	64	87	78	71	75	75	82	72	86	79	85	77
## 90	28-Sep	75	68	80	80	75	78	79	73	78	85	81	85	86
## 91	29-Sep	72	77	75	77	73	71	73	66	78	85	70	81	86
## 92	30-Sep	64	86	75	71	75	71	79	71	80	82	75	79	86
## 93	1-Oct	66	75	86	73	77	75	82	72	82	83	83	80	74
## 94	2-Oct	72	73	78	75	79	80	84	68	82	85	81	82	74
## 95	3-Oct	84	75	77	84	82	81	84	66	80	83	82	77	80
## 96	4-Oct	70	78	82	71	81	80	82	77	81	85	84	80	83
## 97	5-Oct	66	81	82	73	82	79	87	78	80	81	86	81	83
## 98	6-Oct	64	82	73	71	73	70	86	75	75	72	76	82	82
## 99	7-Oct	60	82	82	73	66	68	80	73	75	72	72	83	82
## 100	8-Oct	78	82	69	73	55	79	71	73	73	73	72	83	72
## 101	9-Oct	70	80	72	72	55	66	66	73	71	70	79	81	75
## 102	10-Oct	72	82	73	72	64	73	70	73	71	77	80	81	77
## 103	11-Oct	69	82	78	73	71	75	78	66	77	82	80	67	78
## 104	12-Oct	69	79	78	70	73	78	84	78	73	74	71	72	77
## 105	13-Oct	73	80	78	64	75	78	79	78	64	77	62	74	77
## 106	14-Oct	79	68	75	75	75	75	68	78	63	78	69	78	80
## 107	15-Oct	81	63	79	73	77	75	57	69	62	79	70	78	81
## 108	16-Oct	80	57	78	77	80	62	66	72	71	76	59	76	83
## 109	17-Oct	82	66	77	80	80	60	64	68	75	75	71	82	69
## 110	18-Oct	66	64	78	71	80	64	68	70	73	81	77	77	67
## 111	19-Oct	63	69	82	66	73	71	71	75	68	83	76	76	65
## 112	20-Oct	68	70	75	60	73	75	73	78	71	83	69	75	66
## 113	21-Oct	79	70	73	64	75	79	71	84	73	80	69	78	72
## 114	22-Oct	81	62	63	73	79	80	64	78	73	67	70	72	68
## 115	23-Oct	69	63	63	57	75	81	59	78	70	70	53	81	62
## 116	24-Oct	73	62	72	59	75	79	68	73	73	56	56	59	54
## 117	25-Oct	73	75	75	64	78	73	60	73	78	54	55	61	67
## 118	26-Oct	75	71	79	69	75	64	68	68	79	61	62	68	70
## 119	27-Oct	75	57	79	75	78	51	69	64	81	63	66	67	59
## 120	28-Oct	81	55	79	73	80	55	75	57	78	62	63	70	50
## 121	29-Oct	82	64	78	72	75	63	75	70	75	64	72	62	59
## 122	30-Oct	82	66	82	75	77	72	68	77	78	69	73	67	65
## 123	31-Oct	81	60	79	75	78	71	60	75	82	70	68	71	67
##	2009	2010	2011	2012	2013	2014	2015							
## 1	95	87	92	105	82	90	85							
## 2	90	84	94	93	85	93	87							
## 3	89	83	95	99	76	87	79							
## 4	91	85	92	98	77	84	85							
## 5	80	88	90	100	83	86	84							
## 6	87	89	90	98	83	87	84							
## 7	86	94	94	93	79	89	90							
## 8	82	97	94	95	88	90	90							
## 9	84	96	91	97	88	90	91							

## 10	84	90	92	95	87	87	93
## 11	86	93	95	90	80	85	92
## 12	90	90	95	84	87	90	93
## 13	84	91	97	90	78	89	92
## 14	89	91	90	90	85	90	90
## 15	89	94	80	90	86	86	89
## 16	90	89	85	92	87	83	88
## 17	88	87	87	93	91	86	93
## 18	82	83	89	93	87	82	92
## 19	80	90	94	91	90	85	91
## 20	82	91	91	84	86	76	93
## 21	86	94	92	90	87	82	93
## 22	84	95	94	95	85	83	92
## 23	87	97	92	97	84	88	88
## 24	88	94	92	97	86	87	91
## 25	90	95	90	98	89	88	90
## 26	92	95	94	98	86	89	91
## 27	90	93	94	97	82	92	92
## 28	89	90	90	97	86	90	94
## 29	85	94	93	94	86	82	93
## 30	82	95	96	96	90	84	94
## 31	85	95	96	88	80	85	93
## 32	89	96	91	94	87	81	89
## 33	83	84	96	99	89	84	94
## 34	90	92	97	94	88	88	94
## 35	92	95	85	87	90	90	97
## 36	92	93	96	90	88	89	95
## 37	89	93	93	86	88	92	88
## 38	91	91	93	84	86	95	88
## 39	92	93	94	92	83	90	92
## 40	93	94	91	88	89	89	93
## 41	93	94	95	87	90	86	94
## 42	95	95	94	85	90	83	91
## 43	86	95	95	88	90	88	90
## 44	90	96	95	91	89	84	89
## 45	90	89	94	88	83	85	90
## 46	90	90	88	85	73	87	90
## 47	88	90	90	91	67	88	90
## 48	87	91	92	87	66	89	89
## 49	88	93	94	87	77	89	88
## 50	90	92	96	84	82	86	89
## 51	88	93	93	84	84	89	88
## 52	88	93	94	88	84	92	89
## 53	85	94	98	84	88	93	92
## 54	81	93	92	88	90	93	87
## 55	86	90	93	86	84	88	89
## 56	87	89	95	85	82	84	84
## 57	90	90	99	90	82	86	86
## 58	83	89	95	90	86	88	85
## 59	75	87	95	80	90	91	83
## 60	86	84	93	86	92	92	81
## 61	79	85	90	80	87	88	74
## 62	79	89	92	89	90	89	84
## 63	71	90	95	91	90	90	87

## 64	78	91	96	89	84	90	90
## 65	79	92	95	85	90	92	89
## 66	83	84	80	77	89	82	92
## 67	83	85	78	85	89	89	87
## 68	85	90	75	85	88	91	85
## 69	84	91	69	92	88	90	85
## 70	87	93	73	88	91	84	84
## 71	84	92	81	83	90	84	87
## 72	80	94	84	84	89	86	85
## 73	75	96	86	83	89	90	86
## 74	81	89	87	81	90	92	78
## 75	80	86	89	81	87	86	75
## 76	82	91	92	83	82	78	77
## 77	79	91	86	87	84	80	80
## 78	82	89	72	86	89	86	79
## 79	73	95	79	83	79	86	83
## 80	80	93	77	79	78	85	83
## 81	74	92	77	81	84	84	87
## 82	81	96	82	79	86	83	89
## 83	79	95	86	85	73	87	77
## 84	84	92	80	87	82	82	76
## 85	83	91	83	81	82	77	81
## 86	85	88	82	78	71	78	74
## 87	87	93	88	82	67	77	67
## 88	85	76	86	86	78	74	71
## 89	80	81	84	88	79	78	71
## 90	83	76	79	86	77	74	75
## 91	72	79	84	84	76	71	77
## 92	74	76	78	72	77	84	85
## 93	76	79	65	75	82	86	71
## 94	75	78	68	72	82	85	66
## 95	76	68	75	74	82	78	66
## 96	74	67	80	82	85	65	70
## 97	62	70	83	82	84	71	73
## 98	71	73	81	83	84	78	76
## 99	79	81	79	68	74	82	81
## 100	80	82	78	63	72	86	82
## 101	85	85	72	70	76	86	81
## 102	74	86	68	73	80	86	71
## 103	77	86	65	75	79	86	73
## 104	66	80	73	79	81	85	76
## 105	73	80	74	75	82	85	81
## 106	66	73	77	77	77	75	78
## 107	61	78	80	77	68	69	81
## 108	61	76	84	74	74	70	77
## 109	51	80	85	75	72	80	70
## 110	55	78	80	74	73	76	66
## 111	61	82	67	73	63	73	64
## 112	68	77	59	71	70	73	71
## 113	71	80	63	76	72	77	76
## 114	74	78	68	79	69	70	79
## 115	72	76	70	78	63	72	81
## 116	69	81	73	79	66	74	76
## 117	65	76	76	80	56	77	71

```
## 118 65 85 77 80 61 84 67
## 119 60 76 79 70 69 84 56
## 120 71 74 74 56 64 77 78
## 121 75 68 59 56 75 73 70
## 122 66 71 61 56 78 68 70
## 123 69 75 65 65 74 63 62
```

Using July through October daily-high-temperature data for Atlanta for 1996 through 2015, use a CUSUM approach to identify when unofficial summer ends (i.e., when the weather starts cooling off) each year

```
cusum = function(temps, target = mean(temps), k = 0.3) {
  n = length(temps)
  pos = numeric(n)
  neg = numeric(n)

  for (i in 2:n) {
    pos[i] = max(0, pos[i-1] + temps[i] - target - k)
    neg[i] = max(0, neg[i-1] + target - temps[i] - k)
  }
  list(pos = pos, neg = neg)
}

years = colnames(temps)[-1]

cusum_results = list()

for (year in years) {
  year_data = temps[[year]]
  result = cusum(year_data)
  cusum_results[[year]] = result
}

#Calculating the average of the temperature where the peak is at so that I can use that as my threshold

peak_vals <- sapply(cusum_results, function(x) max(x$pos))
mean_peak <- mean(peak_vals)
h = quantile(peak_vals, probs = 0.15)
h
```

```
##      15%
## 276.1576
```

```
for (year in years) {
  c_pos <- cusum_results[[year]]$pos
  c_neg <- cusum_results[[year]]$neg

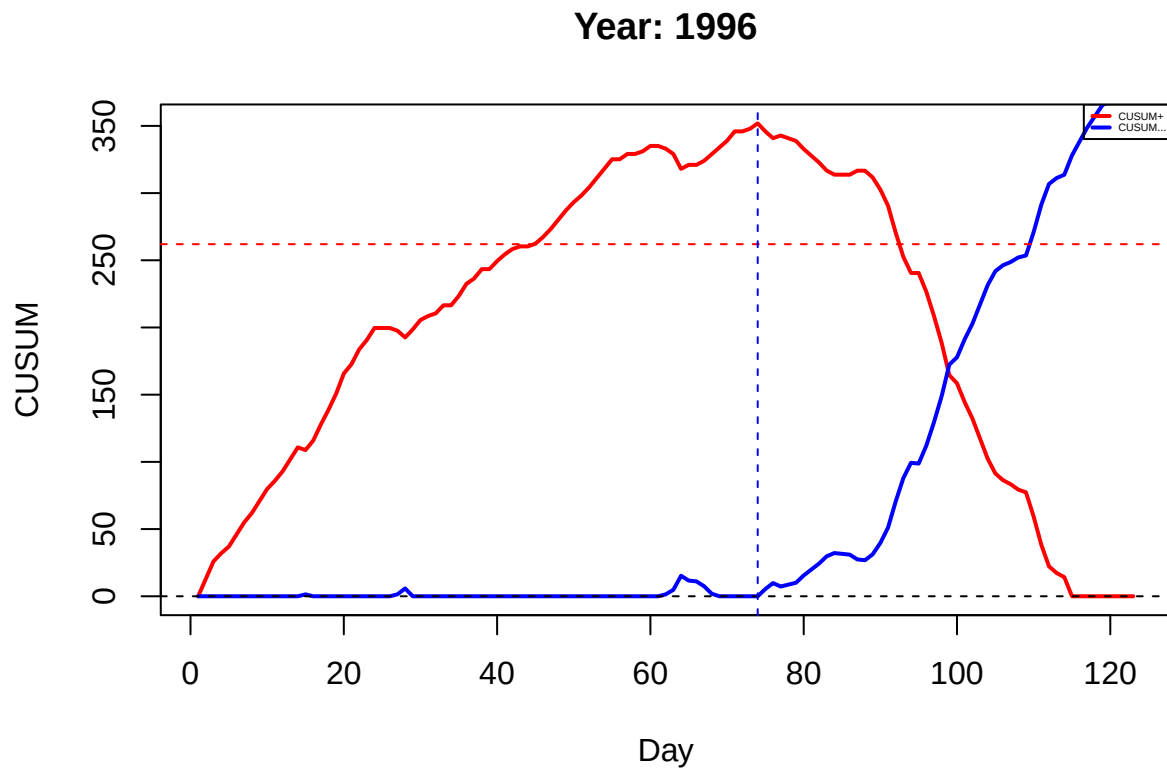
  plot(c_pos, type = "l", col = "red", lwd = 2,
       main = paste("Year:", year),
       ylab = "CUSUM", xlab = "Day",
       ylim = c(min(c_neg), max(c_pos))) # allow full range

  lines(c_neg, col = "blue", lwd = 2)
  abline(h = 0, lty = 2) # baseline
}
```

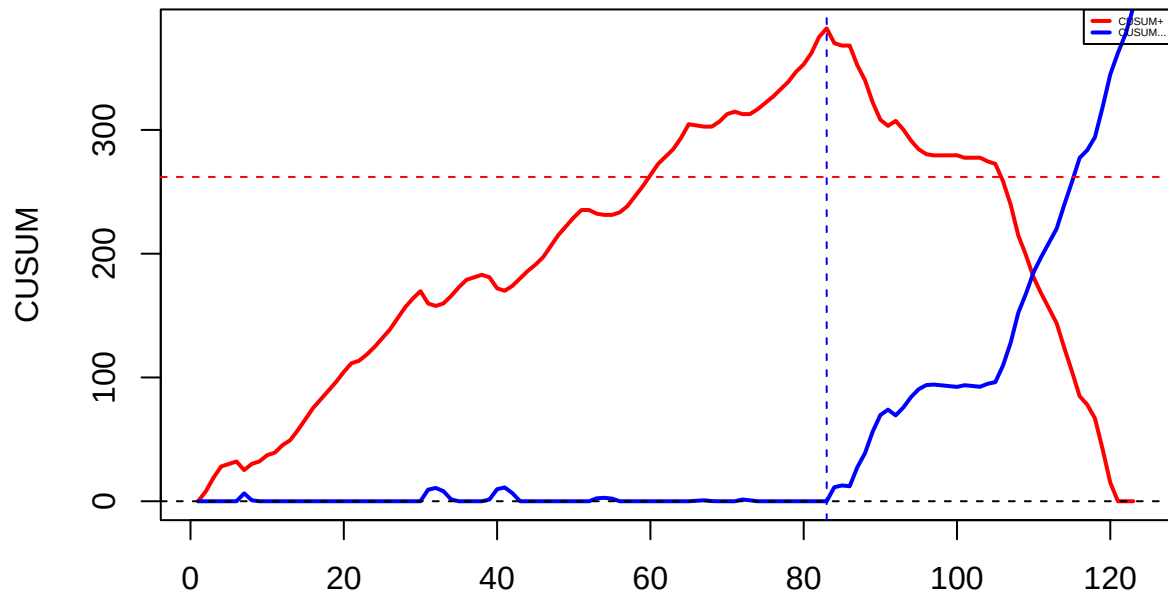
```

legend("topright", legend = c("CUSUM+", "CUSUM-"),
      col = c("red", "blue"), lwd = 2, cex = 0.3)
abline(h = 262, col = "red", lty = 2)
abline(v = which.max(cusum_results[[year]]$pos), col = "blue", lty = 2)
}

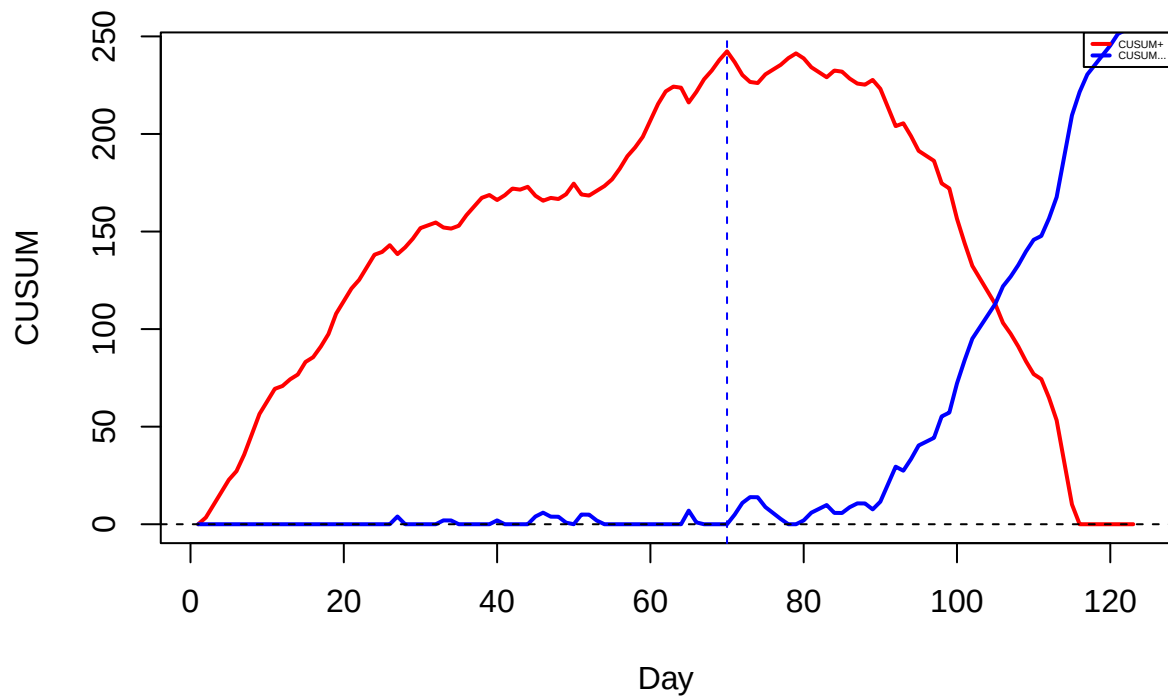
```



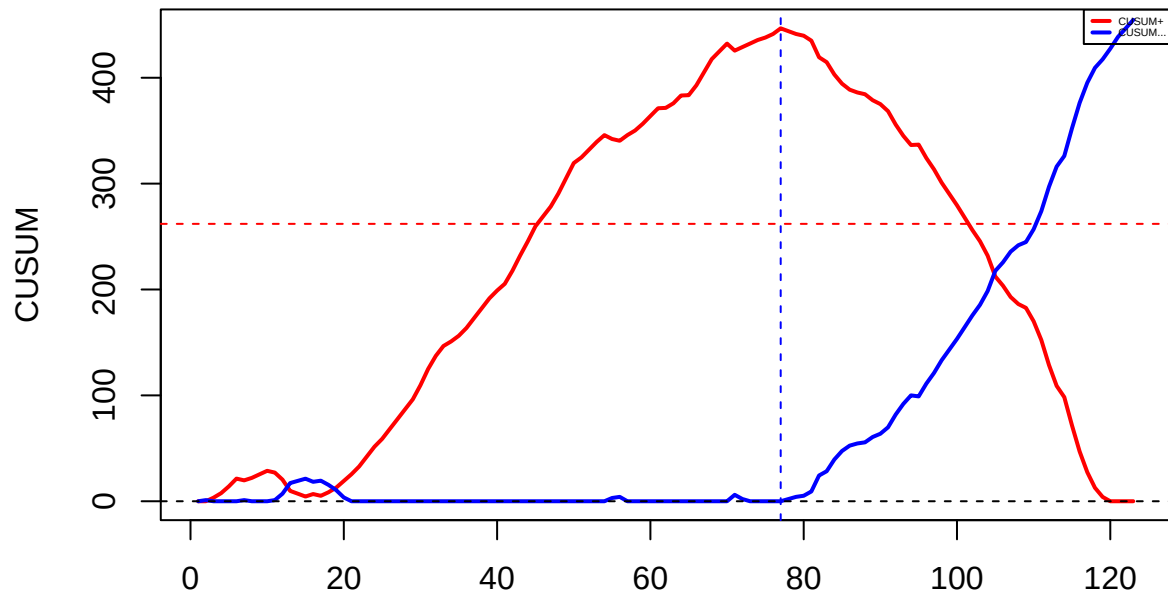
Year: 1997



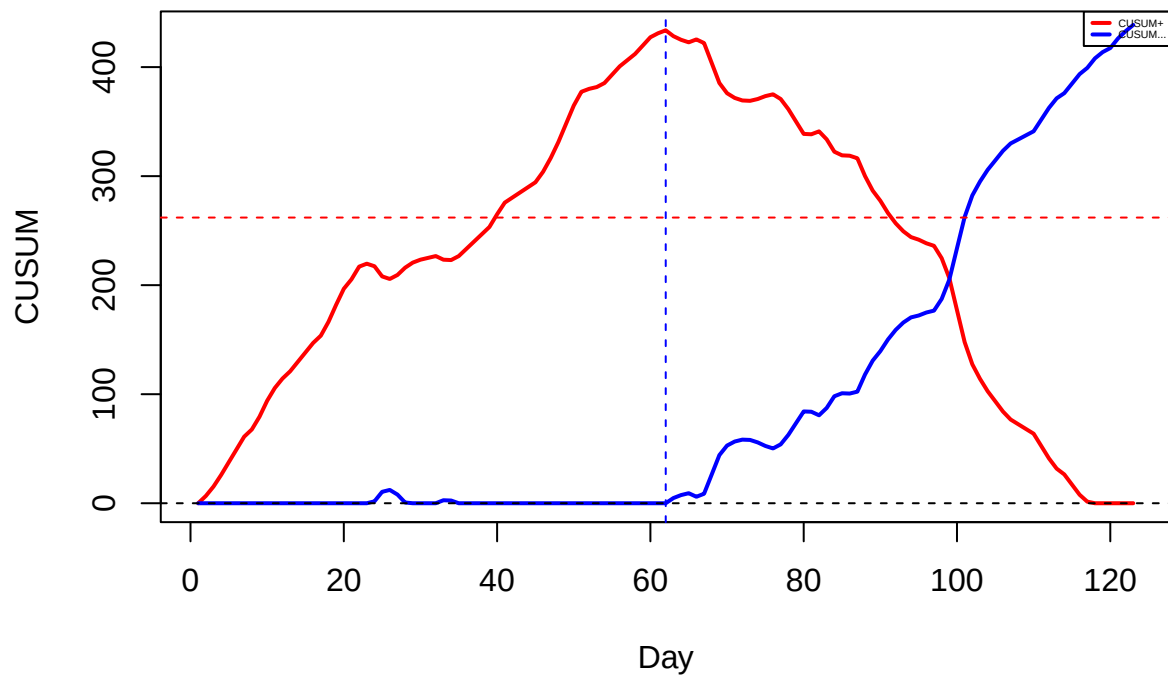
Day
Year: 1998



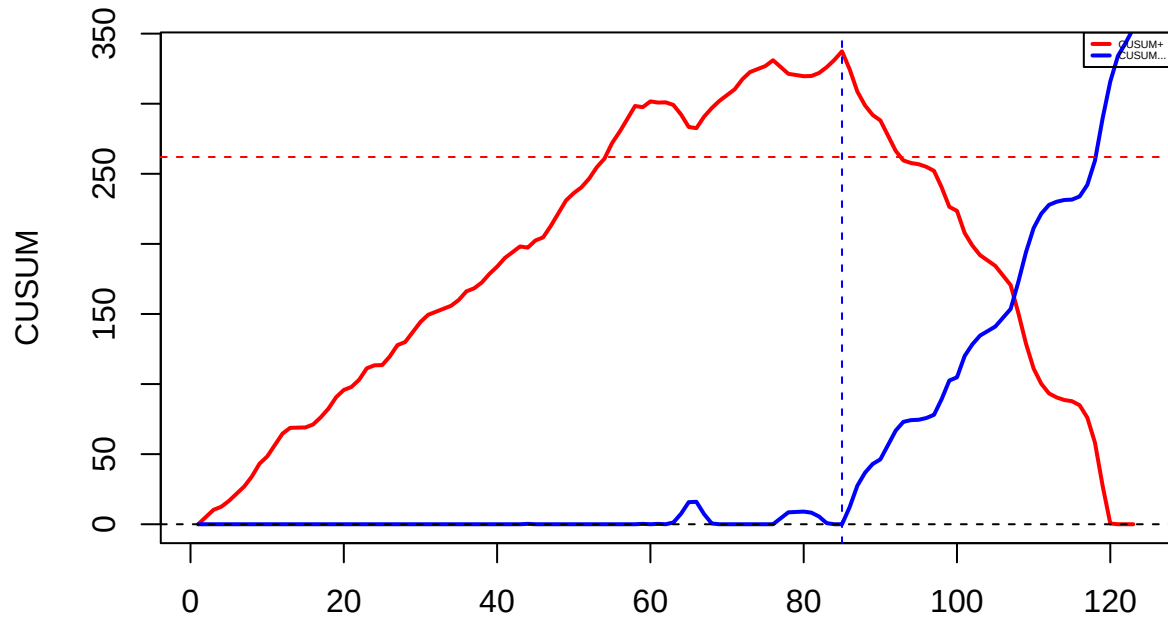
Year: 1999



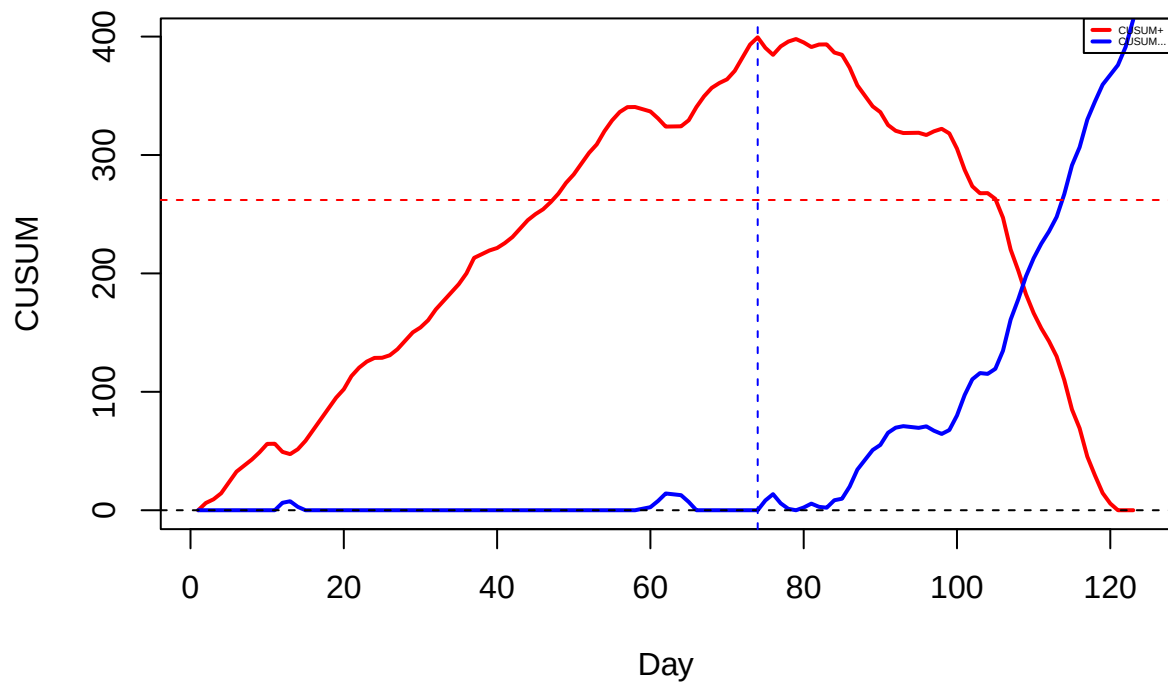
Day
Year: 2000



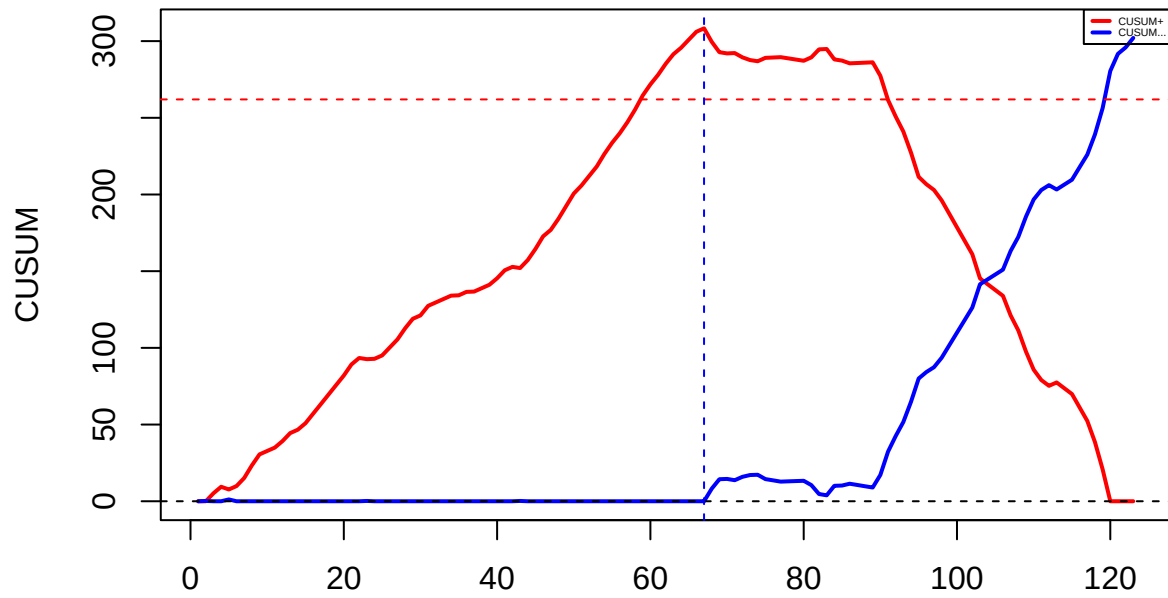
Year: 2001



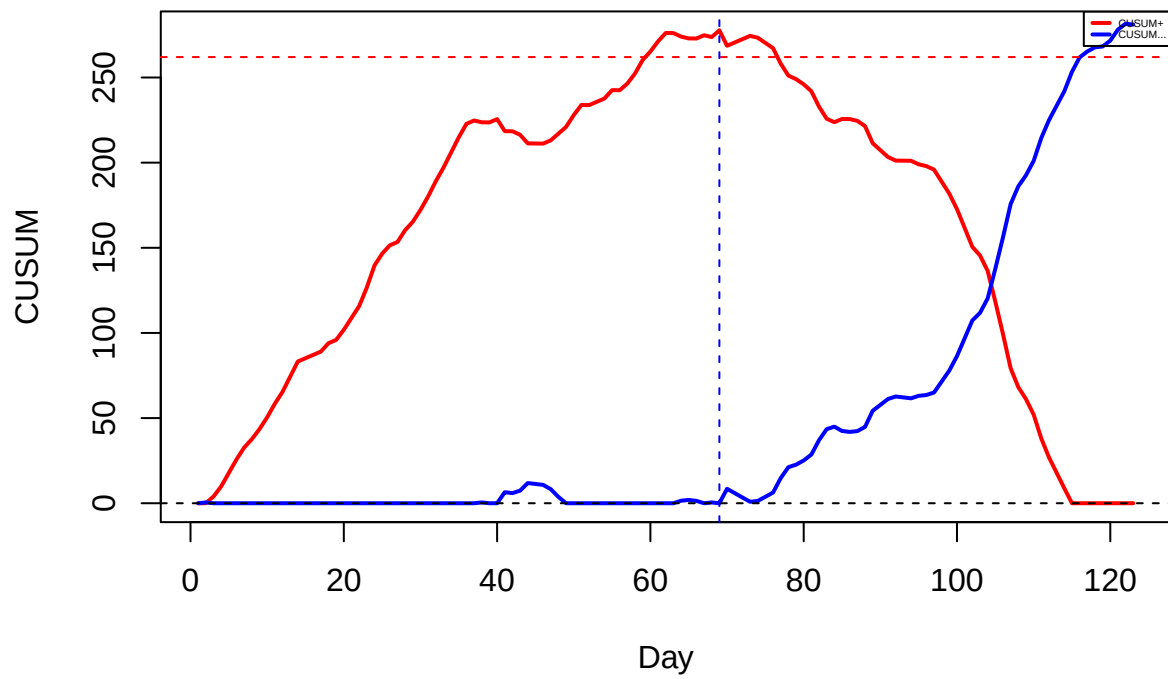
Day
Year: 2002



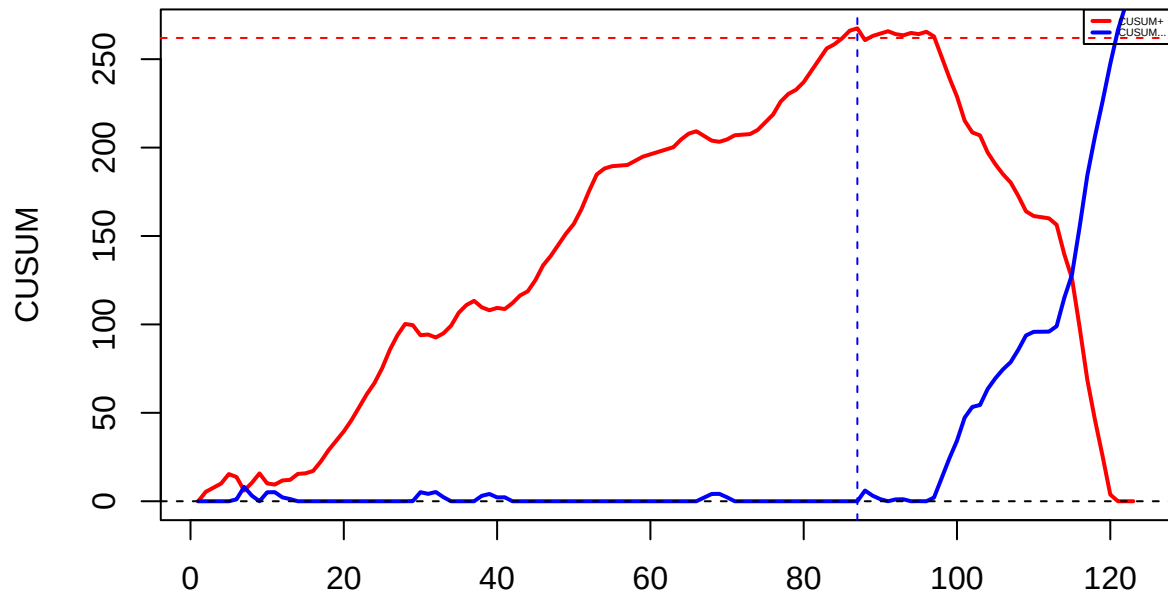
Year: 2003



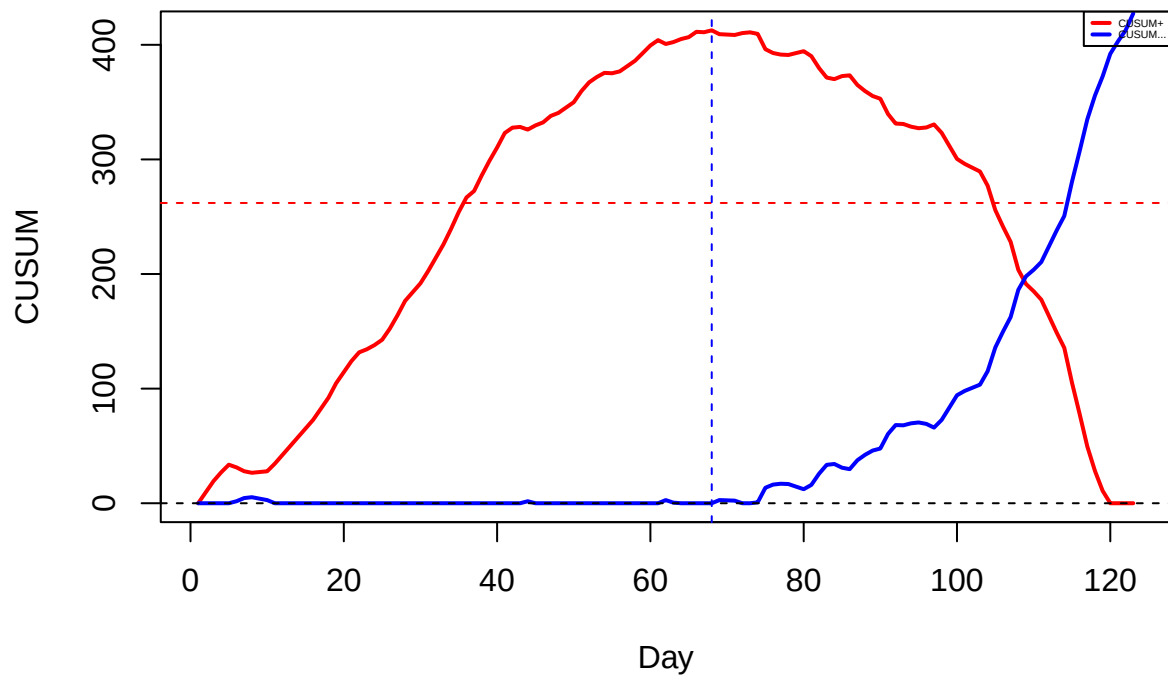
Day
Year: 2004



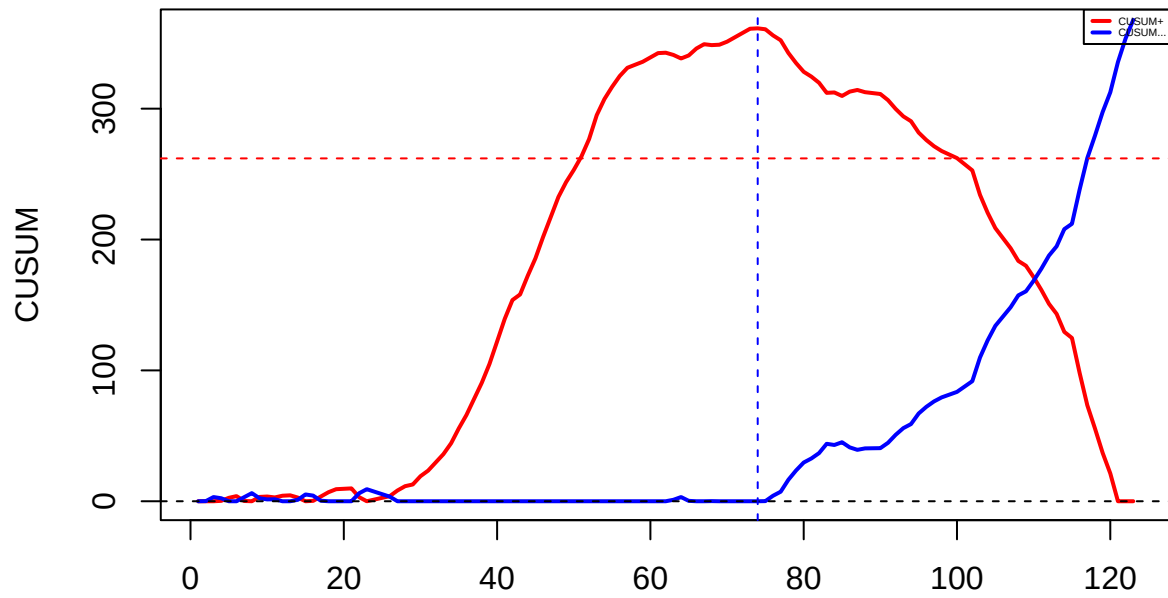
Year: 2005



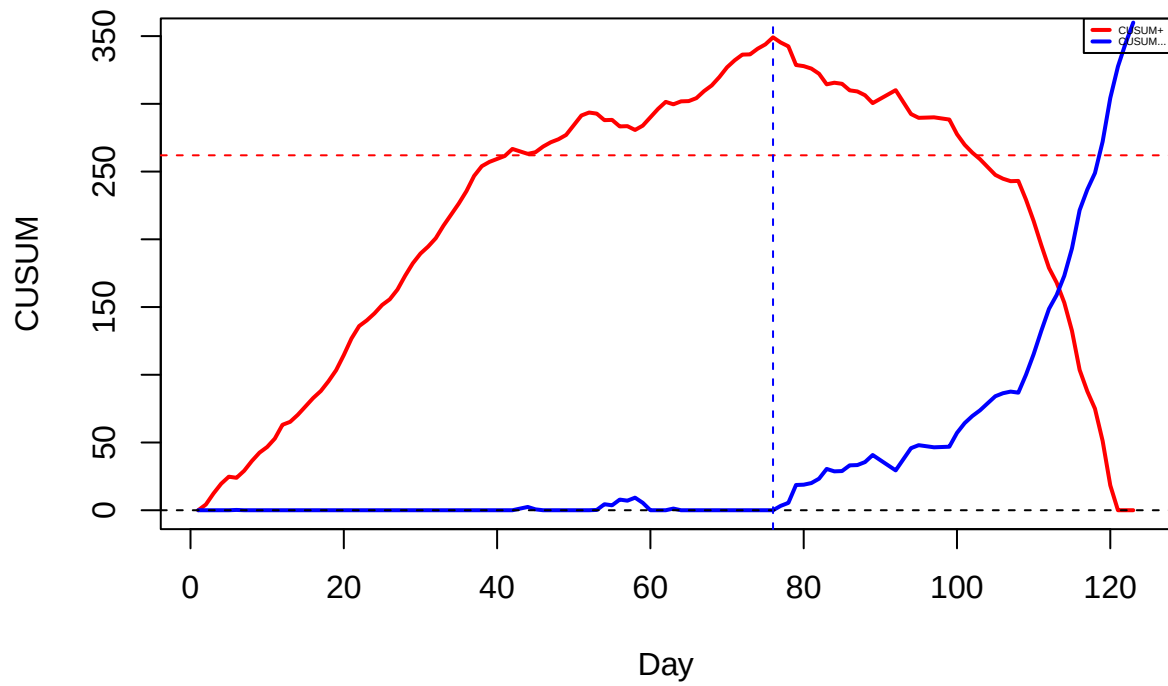
Day
Year: 2006



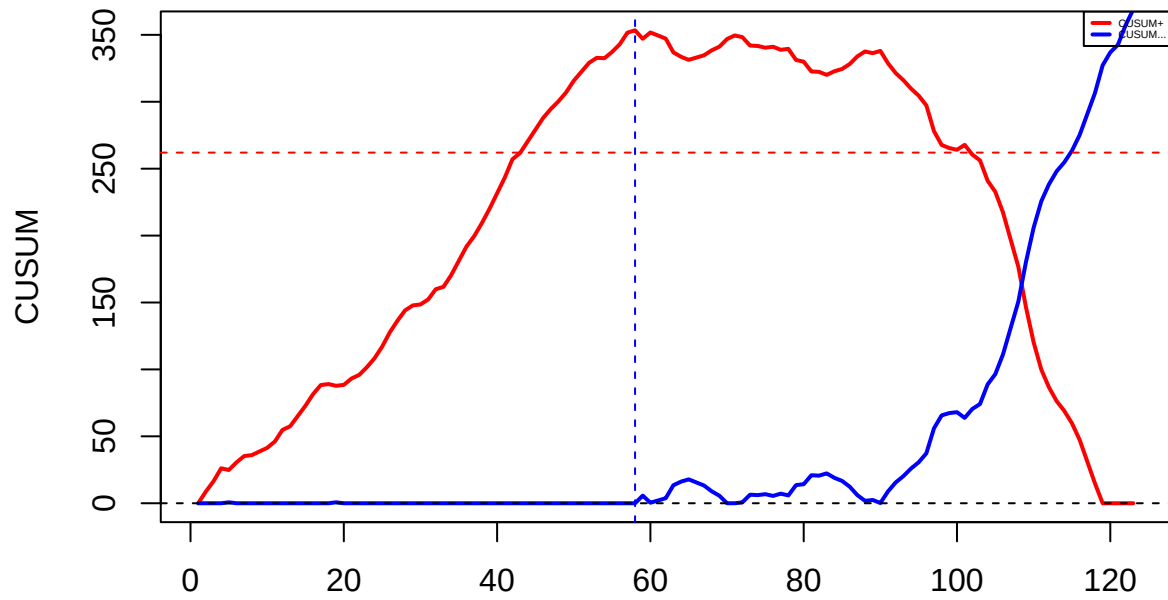
Year: 2007



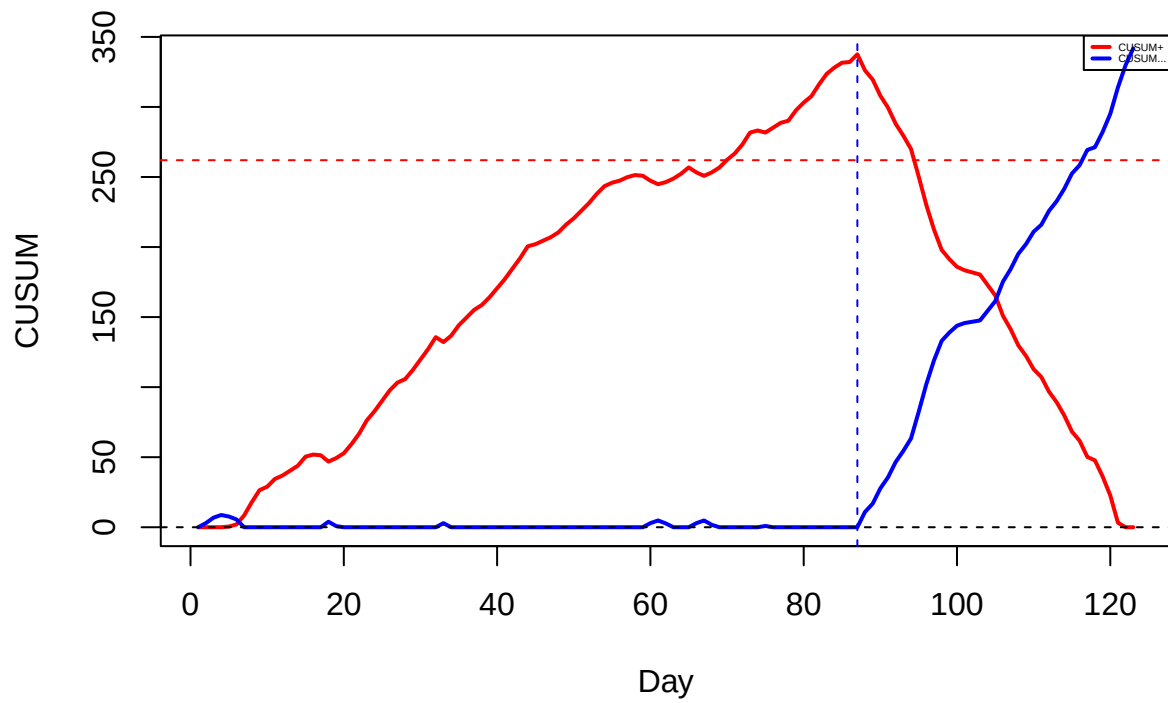
Year: 2008



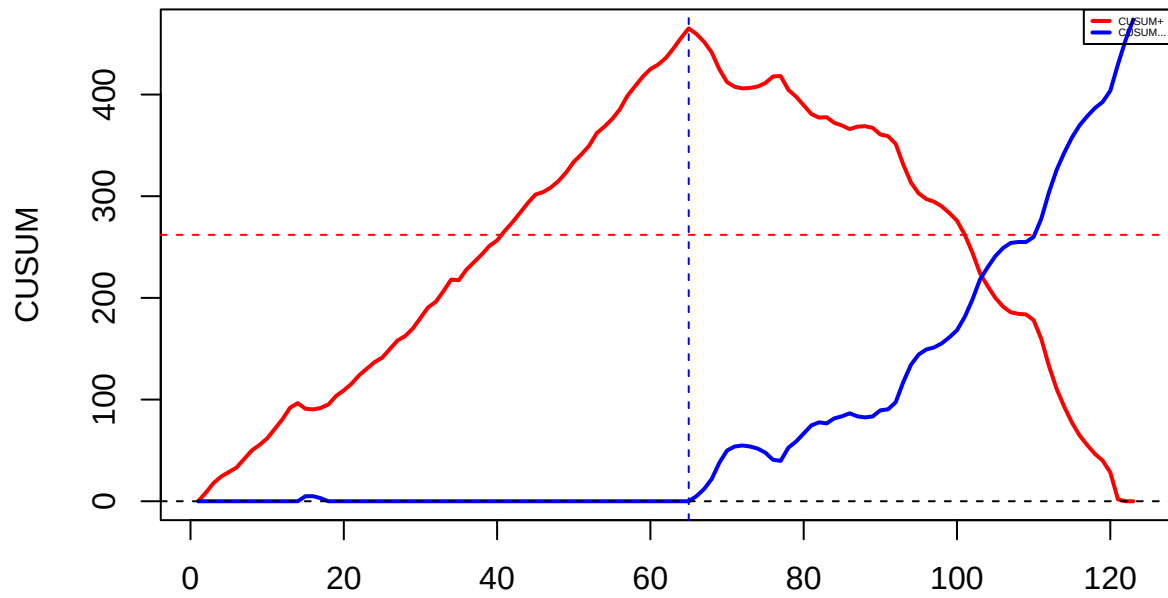
Year: 2009



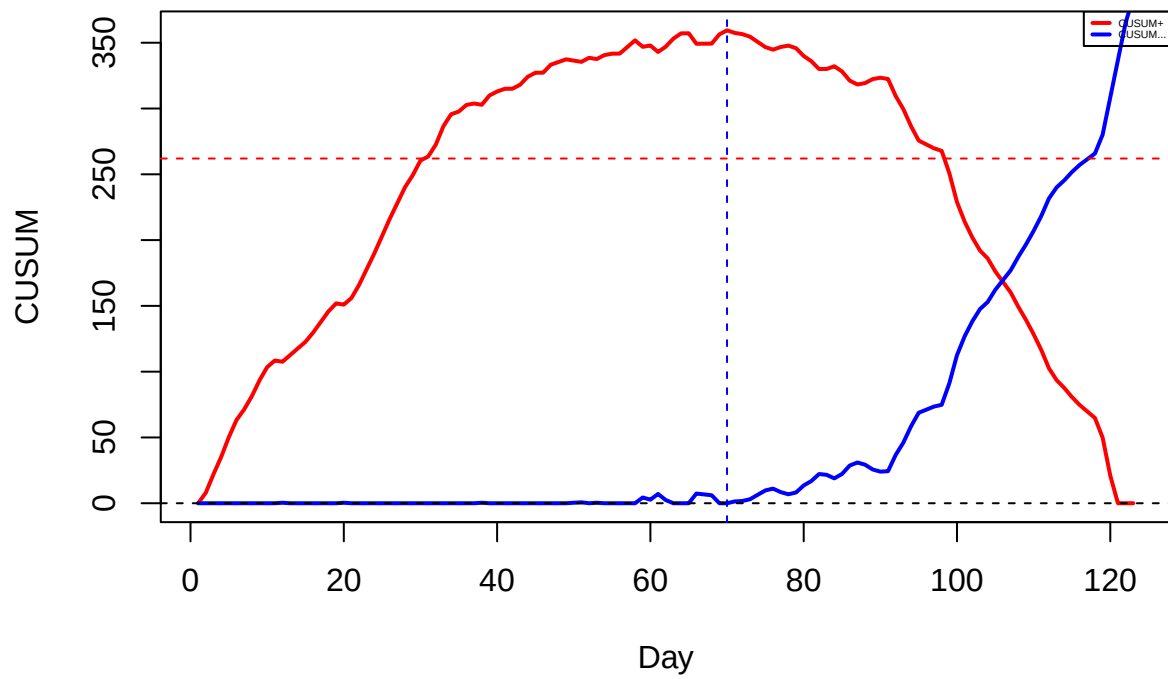
Year: 2010



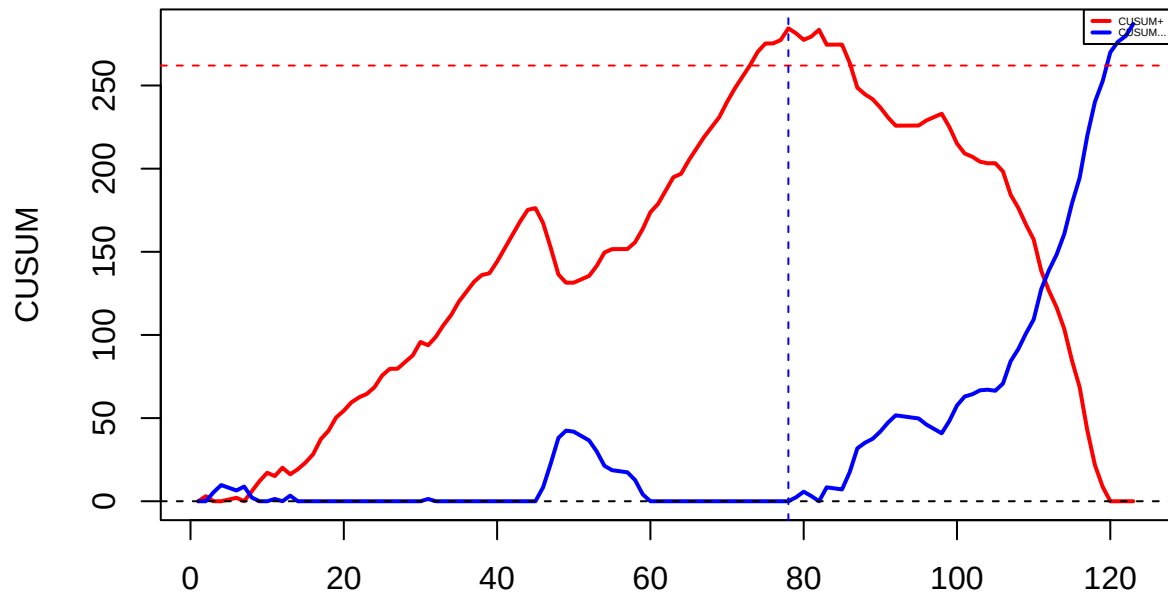
Year: 2011



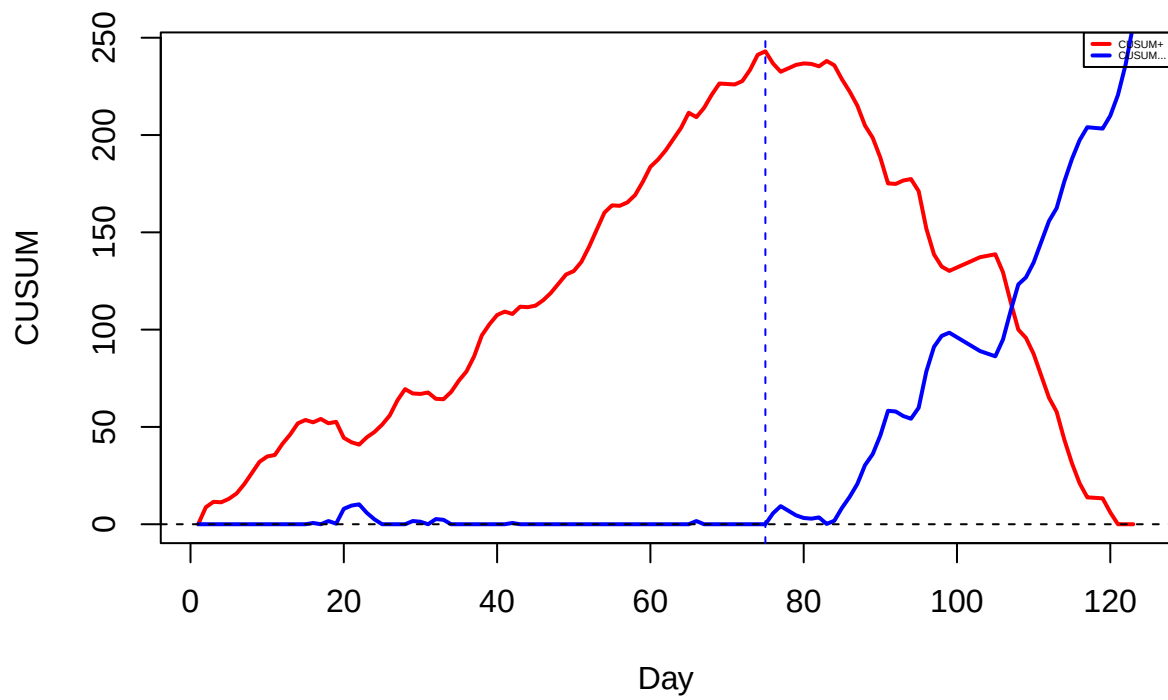
Year: 2012



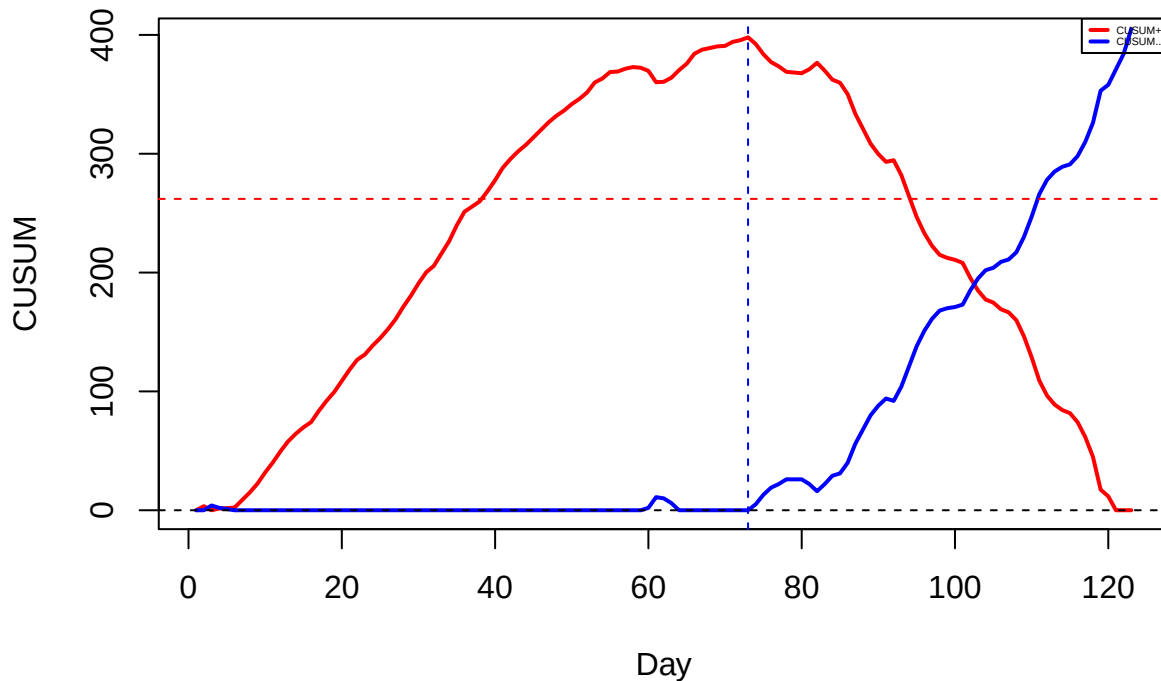
Year: 2013



Year: 2014



Year: 2015



I set my k -value to 0.3. The reason for this is that I did not want to include meaningless deviations that may cause $C+$ to increase. At the same time, since a couple graphs were barely above the threshold, I wanted to keep k at a balanced value so that it showed the clear result of when “unofficial summer” ended. I only wanted to consider the temperature of the days where it was meaningfully above the average. For my threshold, I took the average of the peak values of each year. However, the threshold was still pretty strict, so I took the 15% quantile of that average, and ended up using 262 as my threshold. The x-axis, Days, is a time series that goes through each day of the data. The y-axis, $C+$, represents the accumulated amount of above-average temperatures over time. Since I perform the calculation in my for loop ($\text{temp} - \text{mean} - k$), it created the deviation for me, and then I would add up those deviations to represent the $C+$ accumulation. Once the decrease reached the threshold, that would be the day I would consider the end of “unofficial summer”. Almost every plot dropped below the threshold from Days 80-100. This means that the end of unofficial summer is roughly between September 18th to October 8th. Additionally, the solid blue line shows the increase of the cooling temperatures. In almost every plot, the blue line increases when the red line hits the threshold. This proves that “unofficial summer” is ending. However, there were a couple plots whose max accumulated temperature didn’t reach my threshold. However, I still looked at the plots and noticed the same pattern in the drop of accumulated temperatures and an increase in the $C-$ blue line from September 18th to October 8th.

```
years = colnames(temps)[-1]
summer_average = sapply(temps[, years], mean, na.rm = TRUE)
names(summer_average) = years
```

```
cusum_clim = function(temps, target = mean(temps), k = 0, reset = FALSE) {
  n = length(temps)
  cumsum = numeric(n)
  for (i in 2:n) {
    cumsum[i] = cumsum[i-1] + (temps[i] - target - k)
    if (reset && cumsum[i] < 0) cumsum[i] = 0
  }
}
```

```

    return (cumsum)
}

```

```

cusum_val = cusum_clim(summer_average)

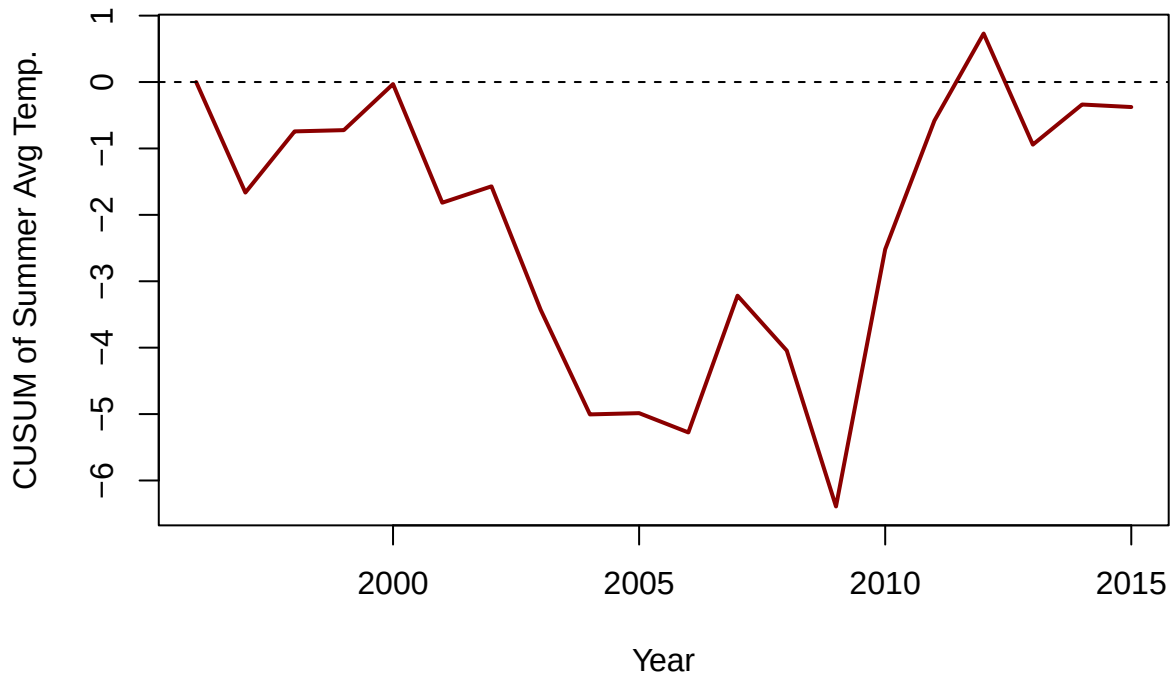
```

```

plot(1996:2015, cusum_val, type = 'l', col = 'darkred', lwd = 2, xlab = 'Year', ylab = 'CUSUM of Summer
abline(h = 0, lty = 2)

```

Detecting Warming in Atlanta Summers



I set $k = 0$ because I want to take all deviations into account. Since we are focusing on the years, small shifts in those annual averages are also meaningful. Therefore, I want to capture all changes. Looking at the graph, the slope suddenly increases starting from 2009. Therefore, the change point is at year 2009. From there on, it strictly increases and peaks at year 2012. The dashed line at $h = 0$ means that there was no cumulative deviation from the mean. When the line is below the dashed line, that means that there is still a cooling trend since the CUSUM is in the negatives. When it is above the dashed line, that means that there is a warming trend. The CUSUM is above $h = 0$ around year 2012. Therefore, there is a clear warming trend that year.

```

n = length(summer_average)
diff_means = numeric(n - 1)

for (i in 1:(n - 1)) {
  before = summer_average[1:i]
  after = summer_average[(i + 1):n]
  diff_means[i] = abs(mean(after) - mean(before))
}

```

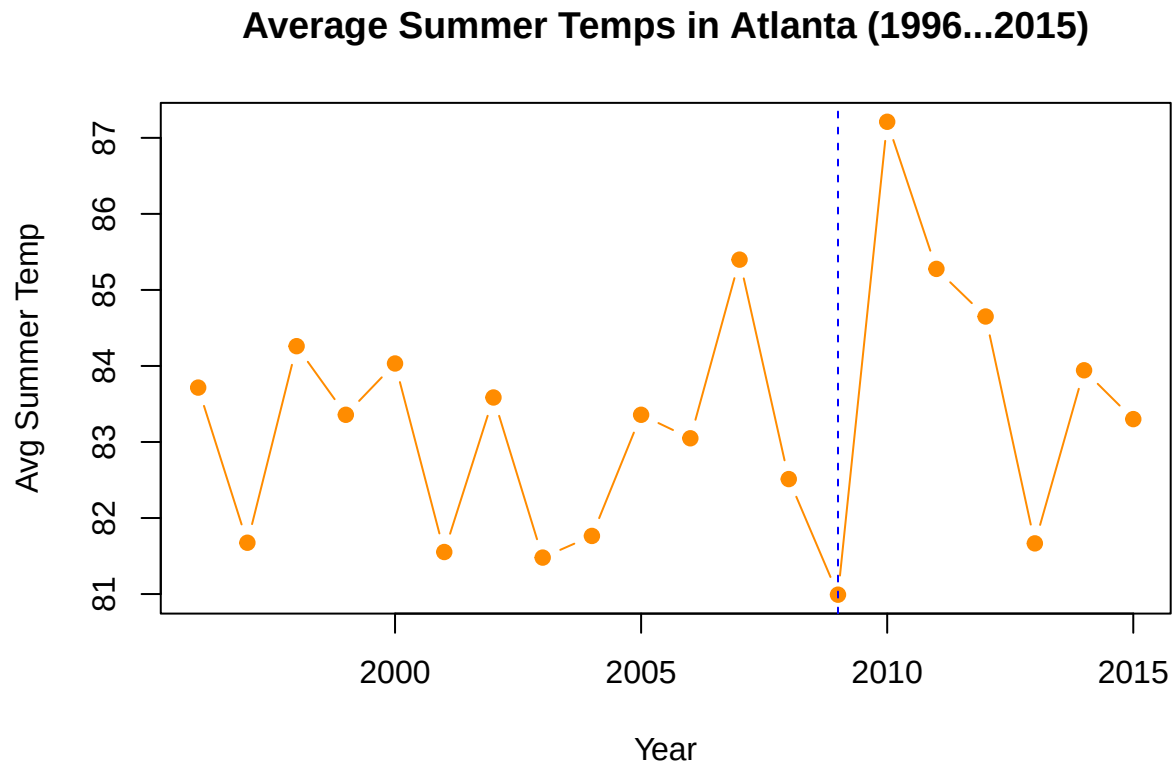
```

cpt_index = which.max(diff_means)
cpt_year = years[cpt_index]

```

```
plot(years, summer_average, type = "b", pch = 19, col = "darkorange",
     xlab = "Year", ylab = "Avg Summer Temp",
     main = "Average Summer Temps in Atlanta (1996-2015)")

abline(v = cpt_year, col = "blue", lty = 2)
```



The graph with the orange line shows the difference in average temperature for each year. The blue line clearly shows that the change point starts at year 2009. There is a huge average difference between 2009 and 2010. It is about a 6 degree difference. After that, the average difference is on a decline.